

2차전지 AI 실무 · 번외 실습

Gemini 에게 말로 시켜 만드는 배터리 품질 분석 앱

프롬프트 14번 · 한 번에 기능 하나
PyQt5 + Bootstrap 사이드바 + Plotly
Random Forest · Isolation Forest · battery_process_data.csv

선수 학습: 2장 파이썬 기초, 3장 전처리, 4장 회귀·분류



학습 목표

이 실습을 마치면

- Gemini 에게 기능을 하나씩 말로 요청해, 메뉴 7개짜리 윈도우 품질 분석 앱을 처음부터 끝까지 키운다
- 목표 화면과 내 화면을 비교하고, 다르면 보이는 대로 한 줄 더 말해 고친다
- 분포 · 상관 히트맵 · 불량 원인 파레토 · 박스플롯 · 스케일링 비교로 데이터를 그림으로 읽는다
- Isolation Forest 이상탐지와 Random Forest 불량 예측이 서로 다른 질문임을 결과로 확인한다
- 혼동행렬과 판정 기준 곡선으로 '정확도는 높는데 불량을 못 잡는' 문제를 찾고 기준을 고른다
- What-if 로 관리 조건을 찾고, 일괄 예측 결과와 HTML 리포트로 현장에 공유한다

오늘 만들 앱 - 배터리 품질 분석기

정답 코드의 실제 화면 (새 로트 예측 → 조건 바꿔 보기)

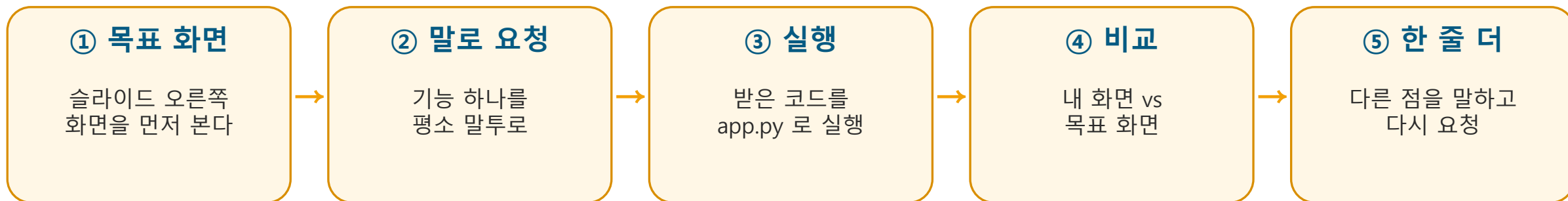


- ≡ 로 사이드바 접기
- 📁 데이터: 열기 · 정제 · 카드
- 📊 분석 차트: 분포 · 히트맵 · 파레토 · 박스 플롯 · 스케일링
- 🚨 이상탐지: Isolation Forest
- 🧠 모델 학습: 카드 · 중요도 · 실제 vs 예측 · 혼동행렬 · 기준 곡선
- 🌐 새 로트 예측: 판정 · What-if
- 📄 일괄 예측 · 📄 리포트

[그림 X-4] 완성 앱 - Gap 110 · RPM 1900 로트 판정과 What-if

진행 방식 - 말로 시키고, 화면으로 확인하고, 다르면 한 줄 더

코드는 Gemini 가, 비교와 조정은 내가



- 프롬프트 14번으로 메뉴가 하나씩 늘어난다: 데이터 → 분석 차트 → 이상탐지 → 모델 학습 → 새 로트 예측 → 일괄 예측 → 리포트
- 7번은 일부러 '불량을 못 잡는' 결과가 나온다 - 8번에서 그걸 보고 고치고, 9 · 10번에서 이유를 그림으로 본다
- 체크포인트 4곳에서 전원이 같은 화면인지 맞춘 뒤 넘어간다
- 막히면 강사용 정답 앱과 화면을 나란히 비교한다

프롬프트 쓰는 요령 - 사람에게 일 시키듯, 한 번에 하나씩

첫 프롬프트로 앱을 시작하고, 이후엔 "~ 추가해 줘 / 반영 바람" 으로 키운다

요령	이렇게 말한다	이러면 곤란
한 번에 기능 하나	분석 차트에 탭 추가해 줘. 히트맵 · 파레토 · 박스플롯	품질 분석 프로그램 전부 만들어 줘
기준은 숫자로	믹싱 온도 10~50 밖이면 결측 / 판정 기준 0.3	이상한 값은 알아서 정리해 줘
보이는 대로 말하기	불량 재현율이 0 이 나옴. 불량률 2% 라 그런 듯	잘 안 되는데 고쳐 줘
모양은 그림으로	(목표 화면 캡처 첨부) 이 화면처럼 바꿔 줘	좀 더 예쁘게 해 줘
에러는 통째로	(오류 메시지 전체 붙여넣기) 실행하니 이 에러 남. 고쳐 줘	에러 나요

[표 X-1] 프롬프트 요령

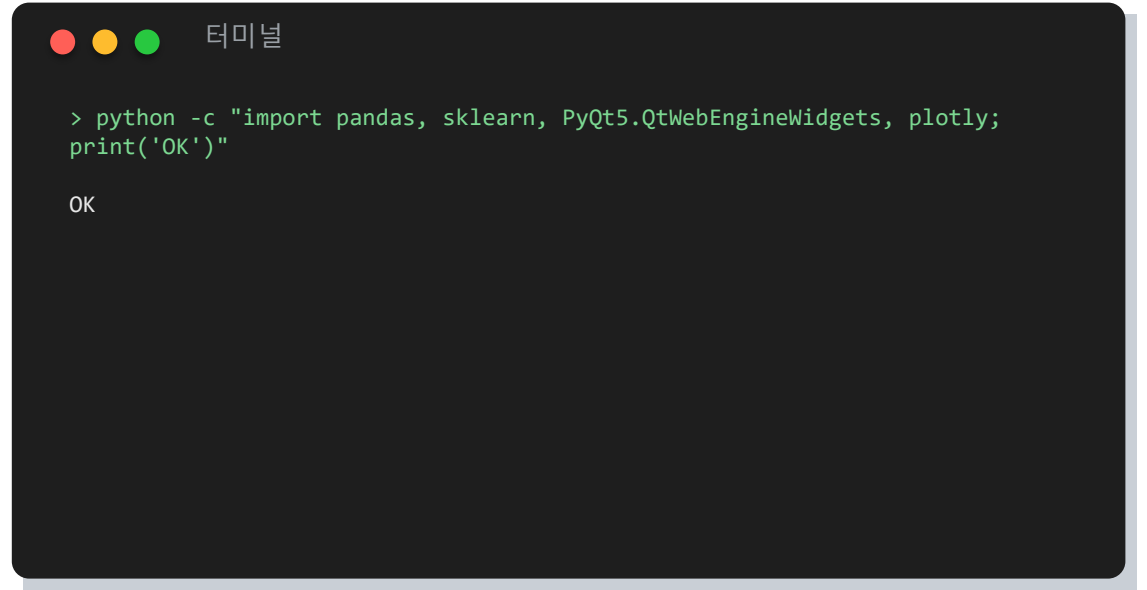
- 대화 창은 하나를 끝까지 - Gemini 가 앞에서 만든 코드를 기억하고 이어서 고친다
- Gemini 가 준 코드 전체를 app.py 에 덮어써서 실행 - 파일 하나로 커져도 괜찮다

번외 실습. Gemini 에게 말로 시켜 만드는 배터리 품질 분석 앱 (PyQt5 + Plotly)

준비물 체크리스트

시작 전에 전원 확인

- Python 3.11+ (Anaconda): pandas · scikit-learn · joblib
- pip install pyqt5 PyQtWebEngine plotly (약 200 MB, 미리 설치)
- 인터넷: plotly 차트 · Bootstrap 은 CDN 에서 불러온다
- Gemini(gemini.google.com) 새 대화 1개 - 파일 첨부 가능
- data/battery_process_data.csv 와 빈 작업 폴더(app.py 하나로 시작)
- 배포 파일: prompts_qt.md (프롬프트 14개 + 목표 화면)



```
터미널

> python -c "import pandas, sklearn, PyQt5.QtWebEngineWidgets, plotly;
print('OK')"

OK
```

배터리 로트 품질 분석기

데이터

정제 완료 → 분석 자료

데이터

CSV 열기

정제

행 수

1000

결측 칸 (정제 전 → 후)

5 → 0

에러 코드 9999

5건

불량 로트

19건

	Lot_ID	막싱_RPM	막싱_온도	코팅_토출압력	코로_1구간_온	프레스_압력	프레스_Gap	비중_용량_mAh	면저항_mOhm	불량_여부
1	L25000	1774	24.9	116.4	115.0	41.0	93.5	3607.4	1428.4	0
2	L25001	1667	23.9	114.7	112.8	43.6	93.2	3603.1	1444.9	0
3	L25002	1810	24.4	123.5	117.9	41.1	90.8	3635.7	1445.3	0
4	L25003	1825	26.0	122.0	103.8	49.0	97.6	3598.3	1545.2	0
5	L25004	1594	25.0	108.8	108.9	45.7	96.4	3539.8	1483.1	0
6	L25005	1646	25.5	129.0	108.1	38.8	85.5	3661.8	1428.2	0
7	L25006	1760	26.0	119.2	102.1	38.7	85.1	3632.3	1332.4	0
8	L25007	1725	24.4	111.1	114.2	41.9	97.5	3559.2	1468.6	0
9	L25008	1749	26.1	129.5	108.9	36.6	90.1	3629.3	1428.9	0
10	L25009	1682	25.6	125.0	113.0	44.7	96.8	3601.8	1524.8	0
11	L25010	1820	23.1	109.9	119.4	41.4	99.7	3544.6	1521.3	0
12	L25011	1812	27.2	134.2	108.6	41.0	87.4	3659.6	1383.8	0
13	L25012	1755	24.2	125.3	108.3	42.7	86.5	3656.5	1385.1	0
14	L25013	1840	22.5	119.9	110.5	46.9	105.0	3552.4	1602.8	0

Random Forest - Plotly

번외 실습. Gemini 에게 말로 시켜 만드는 배터리 품질 분석 앱 (PyQt5 + Plotly)

앱 화면 구조 - 헤더 · ≡ 햄버거 · 사이드바 · 페이지

메뉴가 하나씩 늘어나는 구조 - 프롭트 한 번에 메뉴나 탭 하나



[그림 X-2] 사이드바 펼침 (아이콘 + 글자)



[그림 X-3] ≡ 누르면 접힘 (아이콘만)

부분	PyQt5 에서는	Bootstrap 느낌을 내는 법
헤더 · ≡	QFrame + QToolButton	파란 배경 #0d6efd, 흰 글자
사이드바 메뉴 7개	QPushButton 여러 개	검은 배경 #212529, 선택 메뉴 파랑
페이지 · 탭	QStackedWidget · QTabWidget	메뉴 = 페이지, 페이지 안에서 탭으로 나눈다
KPI 카드 · 버튼	QFrame · QPushButton	흰 카드 · 둥근 모서리 · 파란 버튼을 스타일시트(QSS)로
차트 · 리포트	QWebEngineView	실제 Bootstrap CSS 카드 안에 plotly

프롬프트 1. 앱 시작 - 사이드바와 데이터 표

교재 2.10

Gemini 에 입력

pyqt5, plotly, pandas, scikit-learn 라이브러리를 이
용해서 배터리 공정 품질분석 윈도우 애플리케이션을 만들 거임.

첨부한 battery_process_data.csv 를 열어서 표로
보여 주고, 행 수 · 결측 수 · 에러 코드(9999) 수 ·
불량 건수를 카드로 보여 줘.

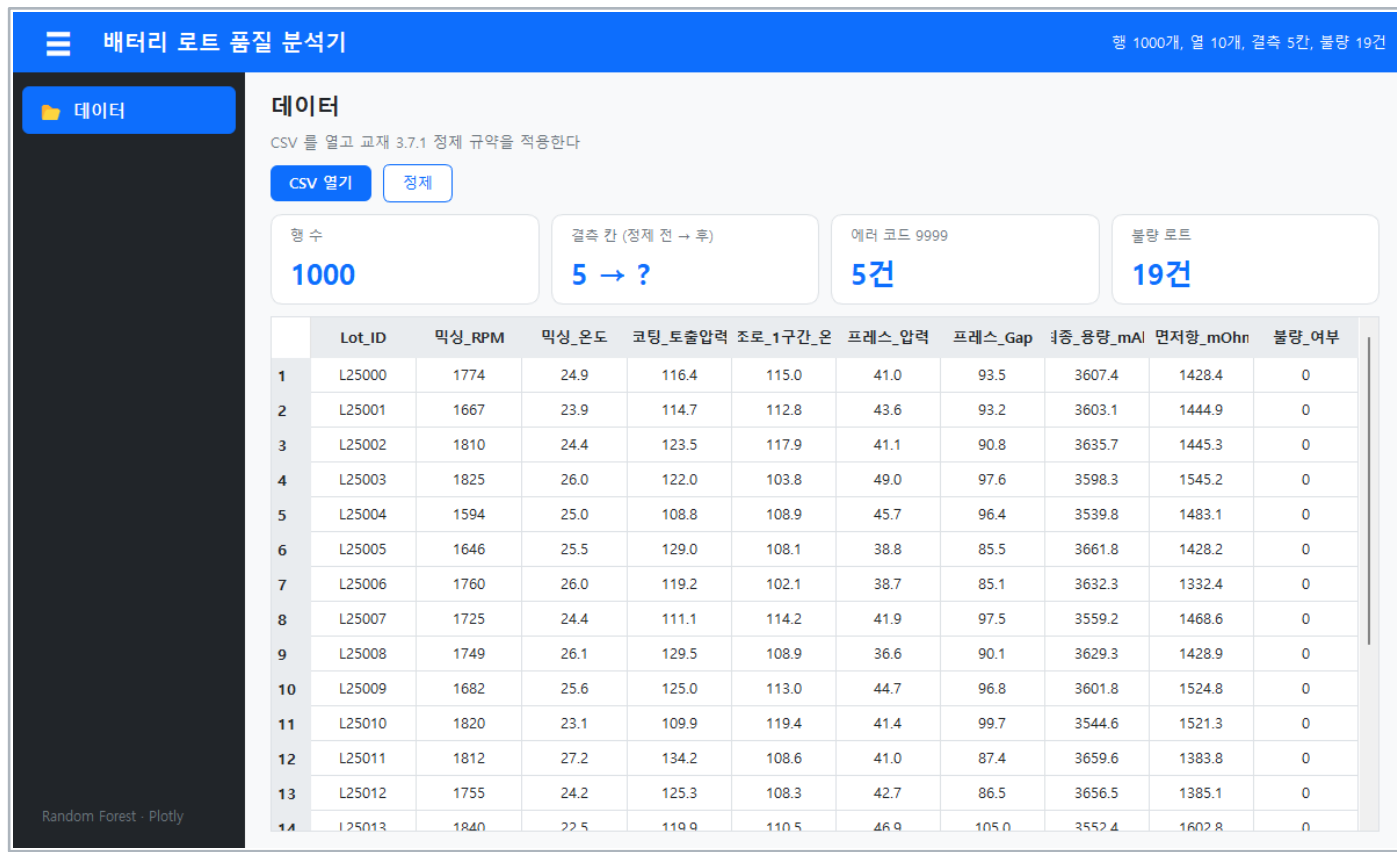
디자인은 부트스트랩 느낌으로, 왼쪽에 햄버거 버
튼으로 접히는 사이드바 메뉴를 넣어 줘.

파이썬 코드 작성 바람.

결과가 다르면 이렇게 한 줄 더

모양이 다르면 이 화면 캡처를 첨부하고 "이 화면처럼
바꿔 줘"

목표 화면 - 이렇게 나오면 성공



읽는 법 사이드바에 '데이터' 메뉴, 카드 4개(1000 · 5 → ? · 5건 · 19건), 아래에 원본 표

프롬프트 2. 데이터 정제

교재 3.7.1

Gemini 에 입력

데이터에 에러 코드 9999 랑 말이 안 되는 값이 섞여 있음.

믹싱 온도 10~50, 건조로 1구간 온도 80~160, 프레스 압력 20~80 밖이면 결측으로 보고, 결측은 중앙값으로 채우는 '정제' 버튼 추가해 줘.
정제 이후 결측 수를 카드에 반영 바람.

결과가 다르면 이렇게 한 줄 더

결측이 0 이 안 되면 "9999 를 먼저 결측으로 바꾼 다음 범위 검사" 라고 순서를 적는다

목표 화면 - 이렇게 나오면 성공

배터리 로트 품질 분석기

정제 완료 → 분석 차트

데이터

데이터

CSV 를 열고 교재 3.7.1 정제 규약을 적용한다

CSV 열기

정제

행 수

1000

결측 칸 (정제 전 → 후)

5 → 0

에러 코드 9999

5건

불량 로트

19건

	Lot_ID	믹싱_RPM	믹싱_온도	코팅_토출압력	조로_1구간_온	프레스_압력	프레스_Gap	1중_용량_mAh	먼저항_mOhm	불량_여부
1	L25000	1774	24.9	116.4	115.0	41.0	93.5	3607.4	1428.4	0
2	L25001	1667	23.9	114.7	112.8	43.6	93.2	3603.1	1444.9	0
3	L25002	1810	24.4	123.5	117.9	41.1	90.8	3635.7	1445.3	0
4	L25003	1825	26.0	122.0	103.8	49.0	97.6	3598.3	1545.2	0
5	L25004	1594	25.0	108.8	108.9	45.7	96.4	3539.8	1483.1	0
6	L25005	1646	25.5	129.0	108.1	38.8	85.5	3661.8	1428.2	0
7	L25006	1760	26.0	119.2	102.1	38.7	85.1	3632.3	1332.4	0
8	L25007	1725	24.4	111.1	114.2	41.9	97.5	3559.2	1468.6	0
9	L25008	1749	26.1	129.5	108.9	36.6	90.1	3629.3	1428.9	0
10	L25009	1682	25.6	125.0	113.0	44.7	96.8	3601.8	1524.8	0
11	L25010	1820	23.1	109.9	119.4	41.4	99.7	3544.6	1521.3	0
12	L25011	1812	27.2	134.2	108.6	41.0	87.4	3659.6	1383.8	0
13	L25012	1755	24.2	125.3	108.3	42.7	86.5	3656.5	1385.1	0
14	L25013	1840	22.5	119.9	110.5	46.9	105.0	3552.4	1602.8	0

읽는 법

결측 5 → 0. 에러 코드 9999 5건이 곧 건조로 범위 위반 5건이다

프롬프트 3. 분포 보기 - 정상 vs 불량

교재 3.9

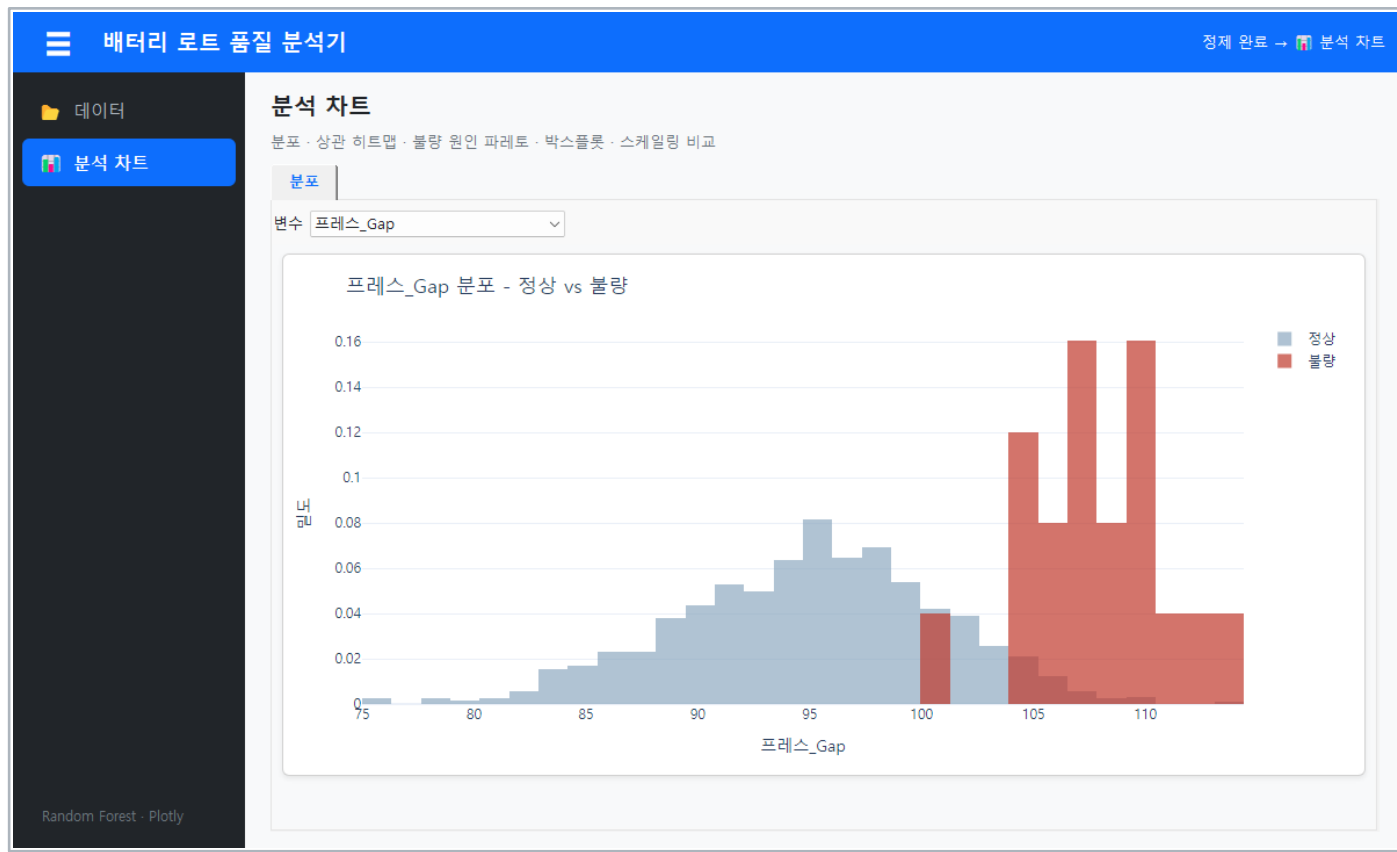
Gemini 에 입력

분석 차트 메뉴 추가해 줘. 차트는 plotly 로.
먼저 '분포' 탭: 변수를 콤보박스로 고르면 정상 /
불량을 겹친 히스토그램으로 보여 줘.

결과가 다르면 이렇게 한 줄 더

불량 막대가 너무 작으면 "불량은 수가 적으니 비율(밀도)
)로 맞추고, 구간은 같게"

목표 화면 - 이렇게 나오면 성공



읽는 법

프레스_Gap 은 불량(빨강)이 105~112 μm 에 몰려 있다 - 정상 분포와 갈리는
변수를 눈으로 찾는다

내 앱의 데이터 메뉴 · 분포 탭과 비교

항목	기대값
행 수	1000
결측 (정제 전 → 후)	5 → 0
에러 코드 9999	5건
불량 로트	19건
분포(프레스 Gap)	불량이 105~112 에 몰림

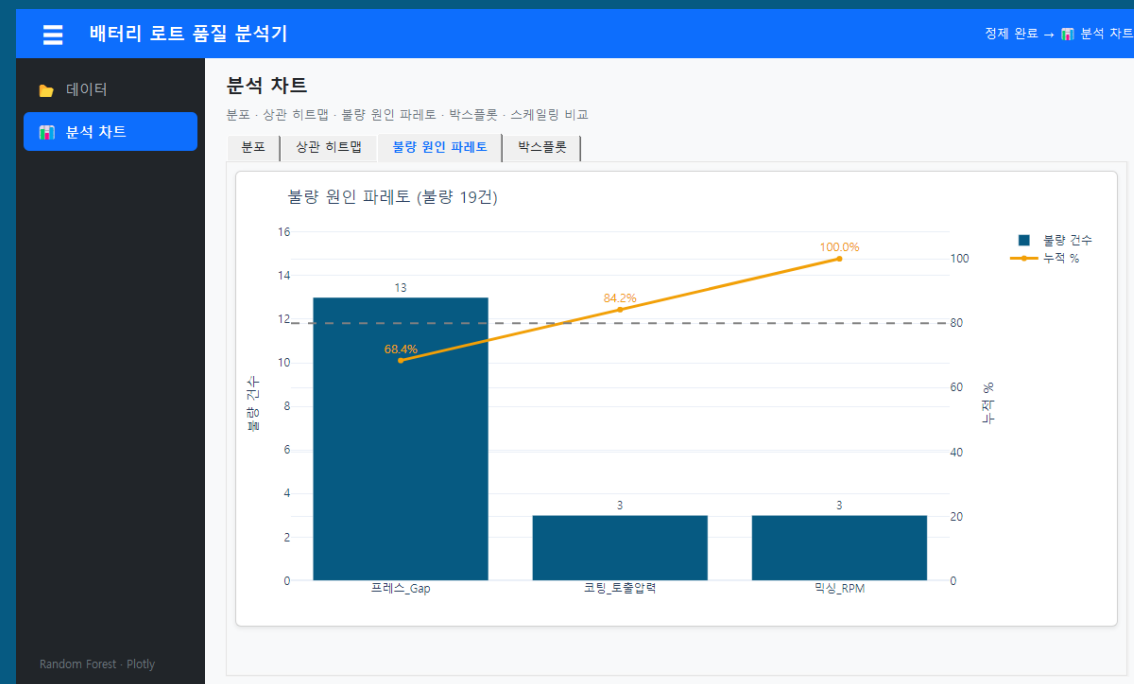
[표] 내 화면과 비교

- **다르면 정제 순서(9999 → 범위 → 중앙값)부터**
- **사이드바 · ≡ 가 없으면 1번 캡처를 첨부**

2부

분석 차트와 이상탐지

프롬프트 4~6
히트맵 · 파레토 · 박스플롯 · 스케일링 · Isolation Forest



프롬프트 4. 상관 히트맵 · 파레토 · 박스플롯

교재 3.11 · 6.8 · 6.9

Gemini 에 입력

분석 차트에 탭 추가해 줘.

- 1) 전체 데이터 상관도 히트맵
- 2) 불량 로트가 어떤 공정 변수 때문인지 파레토 분석
- 3) 정상 / 불량 박스플롯 (변수는 콤보박스로 선택) 반영 바람.

결과가 다르면 이렇게 한 줄 더

파레토 기준이 애매하면 "불량 로트마다 정상 평균에서 가장 많이 벗어난 변수(z-score)를 원인으로 세서"

목표 화면 - 이렇게 나오면 성공



읽는 법

파레토: 불량 19건 중 13건(68%)이 프레스_Gap - 두 변수만 잡아도 84%

분석 차트 읽는 법

숫자 표 대신 그림으로 - 무엇을 먼저 보는가



상관 히트맵

- 진한 빨강 · 파랑 칸만 본다($|r| \geq 0.5$)
- Gap-용량 -0.87: Gap 이 크면 용량이 준다
- 원인이 아니라 '같이 움직임'



파레토

- 왼쪽 막대부터 = 가장 큰 원인
- 누적 % 선이 80% 를 넘는 곳까지가 '핵심 소수'
- 여기서는 Gap · 토출압력 두 개



박스플롯

- 상자 = 가운데 50%, 선 = 중앙값
- 두 상자가 겹치지 않으면 그 변수로 구분된다
- 변수를 바꿔 보면 Gap 만 확실히 갈린다

프롬프트 5. 스케일링 비교 - 단위가 다르면 분포가 어떻게 보이나

교재 3.5

Gemini 에 입력

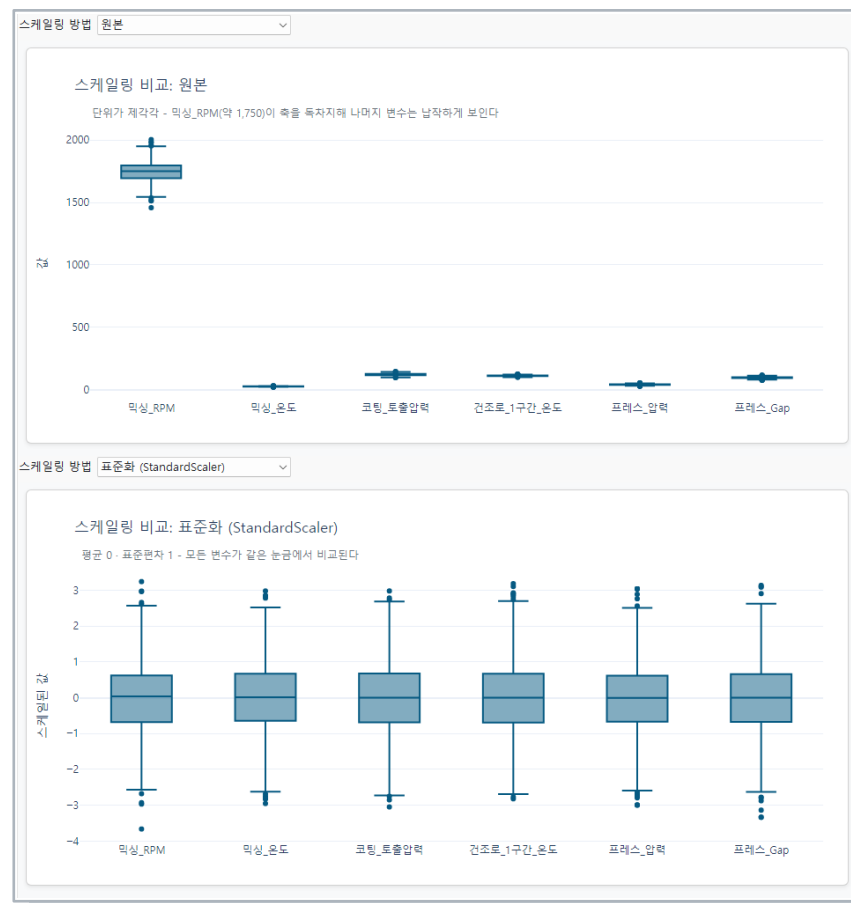
변수마다 단위가 달라서 한 그래프에 그리면 RPM 만 크게 보임.

분석 차트에 '스케일링 비교' 탭 추가해서, 원본 / 표준화 / 정규화(MinMax) / 로버스트 중에 고르면 공정 변수 6개 분포를 박스플롯 하나에 같이 그려서 비교해 줘.

결과가 다르면 이렇게 한 줄 더

변수별로 따로 그려지면 "한 그래프에 상자 6개" 라고 적는다

목표 화면 - 이렇게 나오면 성공



읽는 법

원본은 믹싱_RPM(약 1,750)이 축을 독차지 - 표준화하면 6개가 평균 0 · 표준편차 1 같은 눈금

스케일링 - 같은 데이터, 눈금만 바꿨을 뿐인데

교재 3.5: 단위가 다른 변수를 한자리에서 비교하려면 스케일을 맞춘다



[그림 X-1] 원본 · 표준화 / 정규화 · 로버스트

방법	변환	특징
원본	-	RPM(≈1,750)이 축을 차지, 나머지는 납작
표준화	$(x - \text{평균}) / \text{표준편차}$	평균 0 · 표준편차 1, 가장 흔함
정규화(MinMax)	$(x - \text{최소}) / (\text{최대} - \text{최소})$	0~1, 이상치 하나가 범위를 정한다
로버스트	$(x - \text{중앙값}) / \text{IQR}$	이상치 영향이 작고 벗어난 점이 잘 보인다

- Random Forest 는 스케일에 둔감하다 - 이 실습 모델은 스케일링 없이 학습
- 로지스틱 회귀 · SVM · 거리 기반 이상탐지는 스케일링이 필수
- 스케일러는 학습 데이터로만 fit (교재 3.5 누수 경고)

프롬프트 6. 이상탐지 - Isolation Forest

교재 6.3

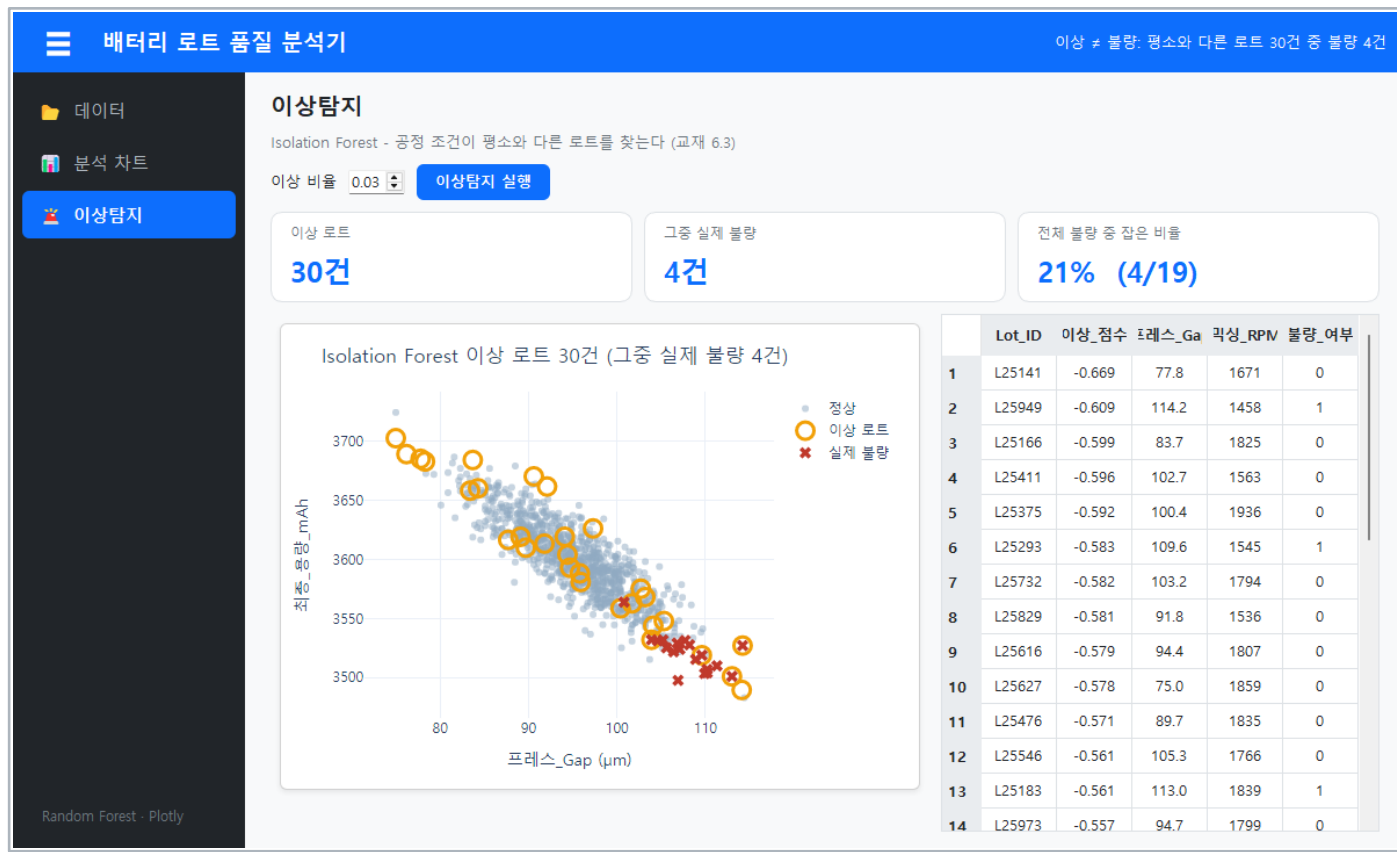
Gemini 에 입력

이상탐지 메뉴 추가.
Isolation Forest 로 공정 조건이 평소와 다른 로트를 찾아서, 이상 로트 수량 그중 실제 불량 수량 몇 건 인지 카드로 보여 주고, 산점도에 동그라미로 표시, 목록은 표로. 이상 비율은 0.03 기본으로 바꿀 수 있게 반영 바람.

결과가 다르면 이렇게 한 줄 더

이상 로트가 0건이거나 너무 많으면 "이상 비율 (contamination) 0.03" 을 적는다

목표 화면 - 이렇게 나오면 성공



읽는 법

이상 로트 30건 중 실제 불량 4건(전체 불량률의 21%) - '평소와 다르다' 와 '불량이다' 는 다른 질문이다

이상탐지와 불량 예측은 다른 질문이다

Isolation Forest 는 정답(불량) 없이 '평소와 다른 로트' 를 찾는다

	이상탐지 (Isolation Forest)	불량 예측 (Random Forest 분류)
묻는 것	이 로트, 평소 공정 조건과 다른가?	이 로트, 불량일까?
정답(불량_여부) 필요	필요 없음 - 비지도	필요 - 지도 학습
이 데이터 결과	이상 30건 중 불량 4건 (불량의 21%)	판정 기준 0.3 에서 불량 4건 중 2건 검출
잘하는 일	처음 보는 조건 · 설비 고장 · 센서 오류 조기 경보	알려진 불량 패턴 재발 판정
현장 쓰임	알람 → 사람이 확인	출하 전 재검 대상 선정

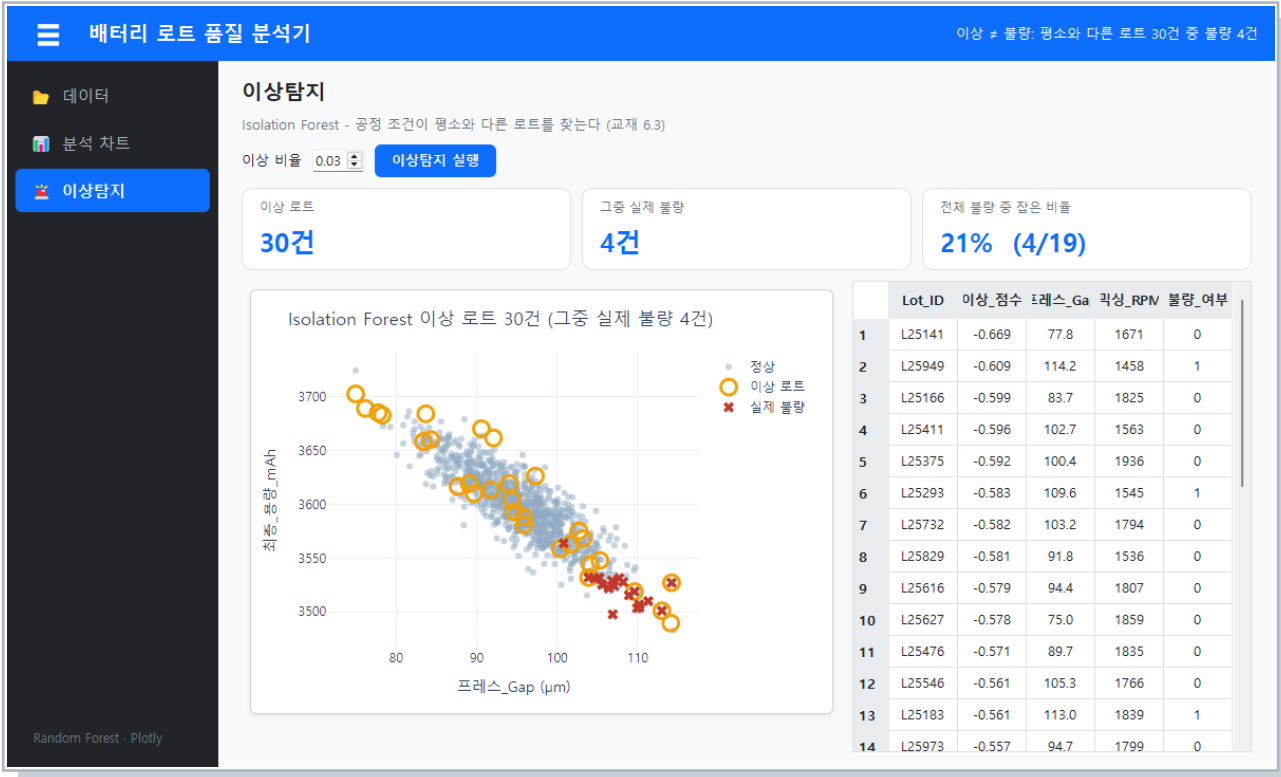
[표 X-2] 이상탐지 vs 불량 예측

- 두 결과가 모두 가리키는 로트(예: L25949 - Gap 114.2, RPM 1458)가 가장 먼저 볼 로트다

번외 실습. Gemini 에게 말로 시켜 만드는 배터리 품질 분석 앱 (PyQt5 + Plotly)

체크포인트 ② - 분석 차트와 이상탐지

탭 5개와 이상탐지 메뉴를 차례로



화면	기대
상관 히트맵	Gap-용량 -0.87
파레토	13 · 3 · 3 (68 · 84 · 100%)
박스플롯	불량 상자가 위
스케일링	표준화 후 6개가 0 근처
이상탐지	30건 / 4건 / 21% (4/19)

[표] 내 화면과 비교

- 차트가 빈 칸이면 "탭이 보일 때 그려 줘"
- 한글이 □ 면 "글꼴 Malgun Gothic"

3부

모델 - Random Forest

프롬프트 7~10

학습 → 문제 발견 → 조정 → 평가 그림 · 판정 기준



프롬프트 7. 모델 학습 - Random Forest

교재 4.4 · 4.11 · 5.9

Gemini 에 입력

모델 학습 메뉴 추가해 줘.
Random Forest 로 최종 용량 예측(회귀)이랑 불량 예측(분류) 둘 다 만들고, 데이터는 8:2 로 나눠서 평가.
 R^2 , RMSE, 불량 재현율, F1 을 카드로, 변수 중요도 는 차트로. 학습한 모델은 저장 / 불러오기 버튼으로.

결과가 다르면 이렇게 한 줄 더

숫자가 조금 달라도 괜찮다(버전 차이). 재현율이 0 이면 그대로 다음 단계로

목표 화면 - 이렇게 나오면 성공



읽는 법

용량 R^2 0.785 는 좋은데 불량 재현율 0.00 (0/4) - 불량을 하나도 못 잡는다. 다음 단계에서 고친다

프롬프트 8. 문제 발견 → 조정: 불량 재현율 0

교재 4.12 · 4.13

Gemini 에 입력

불량 재현율이 0 이 나옴. 불량률 2% 밖에 안 돼서 그런 듯.
불균형 보정하고, 불량 판정 기준 확률을 0.5 말고 0.3 으로 낮춰서 반영 바람.

결과가 다르면 이렇게 한 줄 더

그래도 0 이면 "예측 확률(predict_proba)이 0.3 이상이
면 불량" 이라고 꼭 집어 준다

목표 화면 - 이렇게 나오면 성공



읽는 법 불량 재현율 0.00 → 0.50 (2/4). 정확도가 아니라 '불량을 몇 개 잡았나' 를 본다

프롬프트 9. 모델 평가 그림 - 실제 vs 예측 · 혼동행렬

■ 교재 5.2 · 5.3

Gemini 에 입력

모델 학습 메뉴에 탭 추가해 줘.
실제 용량 vs 예측 용량 산점도(대각선 같이 그려서), 불량 판정 혼동행렬. 반영 바람.

결과가 다르면 이렇게 한 줄 더

축이 헛갈리면 "행은 실제, 열은 예측" 이라고 적는다

목표 화면 - 이렇게 나오면 성공



읽는 법

혼동행렬: 불량 4건 중 2건 잡고 2건 놓침, 정상 4건을 불량으로 오판 - 재현율 0.50의 속사정

프롬프트 10. 판정 기준 곡선 - 기준 하나로 성적이 바뀐다

교재 4.13

Gemini 에 입력

판정 기준을 0.05~0.95 로 바꿔 가며 정밀도, 재현율, F1 이 어떻게 변하는지 선 그래프로 보여 주는 탭 추가해 줘.
지금 기준은 세로 점선으로.

결과가 다르면 이렇게 한 줄 더

점이 하나만 찍히면 "0.05 간격으로 전부 계산해서" 를 적는다

목표 화면 - 이렇게 나오면 성공



읽는 법

기준을 내리면 재현율 ↑ 정밀도 ↓ - 0.1~0.25 에서 F1 최대, 0.4 이상이면 불량 을 하나도 못 잡는다

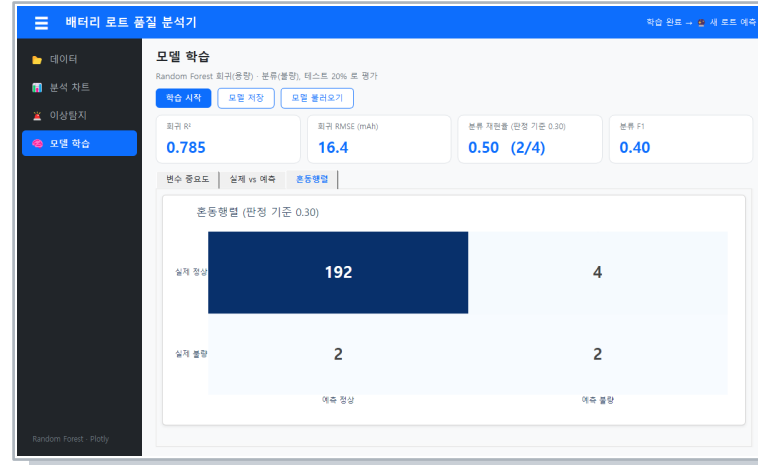
모델 평가 그림 읽는 법

숫자 카드 뒤에 숨은 이야기



실제 vs 예측

- 점이 대각선에 붙을수록 정확
- 위 · 아래로 퍼진 폭 $\approx$ RMSE 16 mAh
- 한쪽으로 치우치면 체계적 오차



혼동행렬

- 행 = 실제, 열 = 예측
- 오른쪽 아래 = 잡은 불량, 왼쪽 아래 = 놓친 불량
- 놓친 불량량이 현장에서 가장 비싸다



판정 기준 곡선

- 기준 $\downarrow \rightarrow$ 재현율 $\uparrow$ 정밀도 $\downarrow$
- F1 이 높은 구간에서 기준을 고른다
- 기준은 비용이 정한다(교재 4.13)

체크포인트 ③ - 모델

소수점은 버전 따라 달라도 된다 - 방향을 본다



카드 · 그림	기대
회귀 R ² / RMSE	0.785 / 16.4 mAh
재현율 (기준 0.5)	0.00 (0/4)
재현율 (기준 0.3)	0.50 (2/4)
혼동행렬 (기준 0.3)	192 · 4 / 2 · 2
중요도 1위	프레스_Gap

[표] 내 화면과 비교

■ 재현율이 그대로면 "확률 0.3 이상이면 불량"

4부

예측과 활용

프롬프트 11~14
새 로트 · What-if · 일괄 예측 · 리포트



프롬프트 11. 새 로트 예측

교재 5.13

Gemini 에 입력

새 로트 예측 메뉴 추가해 줘.
공정 조건 6개를 입력하면 예측 용량, 불량 확률,
판정(정상 / 불량 의심)을 카드로 보여 주고, 판정
기준은 슬라이더로 바꿀 수 있게.
프레스_Gap - 용량 산점도에 예측 위치도 찍어 줘.

결과가 다르면 이렇게 한 줄 더

판정 색이 그대로면 "불량 의심은 빨강, 정상은 초록" 을
추가

목표 화면 - 이렇게 나오면 성공



읽는 법

Gap 110 · RPM 1900 → 3514.4 mAh, 불량 확률 0.513, 빨간 △ 불량 의심 - 별
이 불량(x) 무리 안에

프롬프트 12. 조건 바꿔 보기 (What-if)

교재 6.12

Gemini 에 입력

예측 메뉴에 '조건 바꿔 보기' 탭 추가.
조건 하나를 고르면 다른 조건은 그대로 두고 그 값만 범위 따라 바꿔 가며, 예측 용량과 불량 확률을 그래프로 보여 줘.
판정 기준선도 같이 그려 줘.

결과가 다르면 이렇게 한 줄 더

용량 선이 평평하면 "용량 축은 0 부터 말고 값 범위에 맞춰서"

목표 화면 - 이렇게 나오면 성공



읽는 법

Gap 이 커질수록 용량은 내려가고, 약 106~107 μm 를 넘으면 불량 확률이 판정 기준(0.30)을 넘는다 - 관리 상한 후보

프롬프트 13. 일괄 예측

■ 교재 -

Gemini 에 입력

일괄 예측 메뉴 추가.

CSV 파일 하나를 통째로 예측해서 결과를 예측결과.csv 로 저장하고, 불량 확률 높은 순으로 표에 보여 줘.

결과가 다르면 이렇게 한 줄 더

표 순서가 그대로면 "불량 확률 내림차순으로 정렬"

목표 화면 - 이렇게 나오면 성공

☰ 배터리 로트 품질 분석기

예측결과.csv 저장 (불량 확률 높은 순)

데이터

분석 차트

이상탐지

모델 학습

새 로트 예측

일괄 예측

일괄 예측

CSV 한 파일의 모든 로트를 한 번에 판정하고 예측결과.csv 로 저장한다

CSV 일괄 예측

판정한 로트

1000

불량 의심

21건

	Lot_ID	프레스_Gap	믹싱_RPM	예측_용량_mAh	불량_확률	판정
1	L25949	114.2	1458	3521.3	0.733	불량 의심
2	L25326	107.1	1624	3522.8	0.733	불량 의심
3	L25561	109.9	1777	3507.3	0.73	불량 의심
4	L25293	109.6	1545	3517.8	0.723	불량 의심
5	L25101	104.6	1854	3534.8	0.703	불량 의심
6	L25279	106.9	1771	3512.0	0.7	불량 의심
7	L25289	105.6	1742	3529.2	0.7	불량 의심
8	L25774	110.2	1751	3507.7	0.693	불량 의심
9	L25681	106.4	1800	3529.6	0.683	불량 의심
10	L25827	110.2	1741	3510.9	0.68	불량 의심
11	L25922	111.3	1760	3511.8	0.673	불량 의심
12	L25887	106.8	1715	3534.1	0.667	불량 의심
13	L25253	103.9	1671	3536.8	0.663	불량 의심
14	L25846	100.8	2004	3565.1	0.64	불량 의심

읽는 법

1,000 로트 중 불량 의심 21건 - 위에서부터 재검하면 된다

프롬프트 14. 리포트 내보내기

교재 6.13

Gemini 에 입력

리포트 메뉴 추가.
지금까지 결과(요약 숫자, 파레토 표, 중요도 · 히트
맵 · 파레토 · 실제 vs 예측 그래프)를 HTML 리포트
한 파일로 저장하고, 앱 안에서 미리 보여 줘.
리포트 디자인도 부트스트랩으로.

결과가 다르면 이렇게 한 줄 더

오프라인에서 그래프가 안 나오면 "plotly.js 를 리포트
파일 안에 포함"

목표 화면 - 이렇게 나오면 성공



읽는 법

품질분석_리포트.html 한 파일 - 브라우저만 있으면 열리고 그래프도 그대로 움직인다. 메일로 공유

체크포인트 ④ - 전체 동작

사이드바 메뉴 7개를 처음부터 눌러 본다



메뉴	확인
데이터 · 분석 차트	5 → 0 · 파레토 13·3·3
이상탐지	30건 중 불량 4건
모델 학습	R ² 0.785 · 재현율 0.50
새 로트 예측	⚠ 불량 의심 (0.513)
What-if (Gap)	약 107 μm 에서 기준 초과
일괄 예측	불량 의심 21건
리포트	품질분석_리포트.html
≡	사이드바 접힘

[표] 내 화면과 비교

도전 과제 - 하나 골라 내 앱에 기능 추가

목표 화면은 없다 - 무엇이 나오면 성공인지 스스로 정하고 말로 시킨다

도전	Gemini 에 이렇게 (예시)	성공 기준을 스스로 정해 보기
A. 실행 파일(exe)	이 프로그램을 exe 파일로 만들고 싶어. 방법이랑 명령어 알려 줘	다른 PC 에서 파이썬 없이 뜨나?
B. SPC 관리도 (교재 6.10)	분석 차트에 건조로 온도 X-bar 관리도 탭 추가해 줘. 관리한계선 같이	관리한계를 벗어난 구간이 있나?
C. 다크 모드	헤더에 다크 모드 전환 버튼 넣어 줘. 차트도 같이 어둡게	모든 페이지 · 차트가 같이 바뀌나?
D. 나만의 기능	(현장에서 필요했던 기능 한 가지)	동료에게 보여 줄 수 있는 화면인가?

[표 X-3] 도전 과제

- 마무리: 1~2명이 화면 공유 - 어떤 말로 시켰고, 어디서 한 줄 더 했는지 이야기한다

막혔을 때 - 이렇게 말하면 고쳐 준다

오류 메시지는 통째로, 화면 문제는 보이는 대로

상황	Gemini 에 이렇게
실행하자마자 오류	(오류 메시지 전체 붙여넣기) 실행하니 이 에러 남. 고쳐서 전체 코드 다시 줘
ImportError: QtWebEngineWidgets must be imported before...	QtWebEngine 을 QApplication 만들기 전에 import 하게 고쳐 줘
차트 자리가 흰 빈 칸	탭을 누르면 차트가 빈 칸으로 나옴. 탭이 보일 때 그리게 고쳐 줘
ModuleNotFoundError: PyQt5.QtWebEngineWidgets	(직접) pip install PyQtWebEngine
버튼 · 카드가 회색 기본 모양	부트스트랩 느낌이 안 남. 이 캡처처럼 파란 버튼 · 흰 카드로 스타일 바꿔 줘
≡ 를 눌러도 그대로	햄버거 누르면 사이드바가 아이콘만 남게 접히도록 해 줘
열 이름이 영문으로 바뀜	열 이름은 CSV 에 있는 한글 그대로 써 줘
앞에서 만든 메뉴가 사라짐	방금 코드에서 이상탐지 메뉴가 없어졌음. 기존 메뉴는 그대로 두고 이것만 추가해 줘

[표 X-4] 자주 막히는 곳

Gemini 사용 시 주의 - 만든 앱의 숫자는 내 책임

회사 데이터는 올리지 않는다, 숫자는 교재로 검증한다

- 실제 공정 데이터 · 설비 태그 · 레시피는 첨부하지 않는다 - 이 실습은 공개 실습 CSV 만 쓴다
- 그럴듯한 앱이 조용히 틀린 숫자를 낼 수 있다 - 목표 화면의 숫자와 반드시 비교
- 버전 차이로 소수점은 달라도 된다 - 순위 · 증감 · 판정 방향이 같은지 본다
- Gemini 가 멋대로 바꾸기 쉬운 것: 열 이름 영문화, 판정 기준 0.5, PyQt6, 앞 메뉴 삭제 - 한 줄로 되돌린다
- 한 번에 기능 하나 - "전부 만들어 줘" 는 고칠 수 없는 덩어리를 만든다

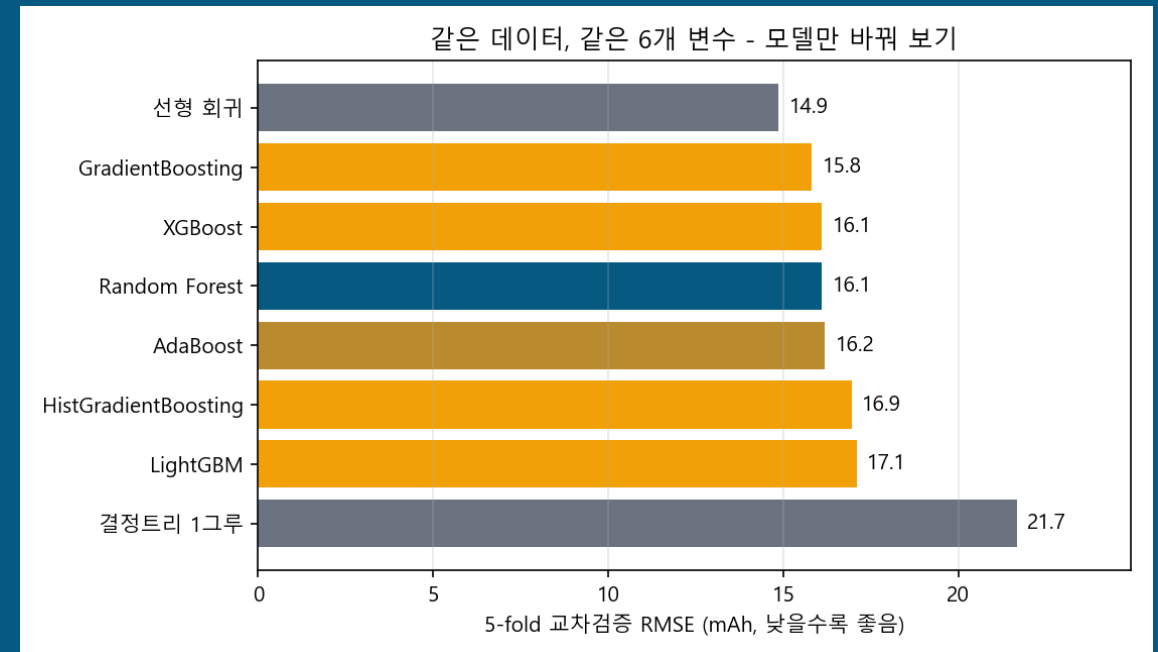
Key Takeaways

- 앱은 한 번에 만들지 않는다 - 기능 하나씩 말로 요청하고 화면으로 확인한다
- 다른면 보이는 대로 한 줄 더: "재현율이 0 이 나옴" 이 가장 좋은 프롬프트다
- 파레토: 불량 19건 중 13건이 프레스_Gap / What-if: 약 107 μm 를 넘으면 불량 위험
- 이상탐지(평소와 다름)와 불량 예측(불량일까)은 다른 질문 - 둘 다 가리키는 로트부터
- 스케일링: 같은 데이터도 눈금을 맞춰야 비교가 된다(교재 3.5)
- 다음: 내 공정 데이터 구조로 같은 순서를 밟아 본다(사내 규정 안에서)

이론

앱 속 숫자 뒤의 이론

평가 지표 - 왜 R^2 보다 RMSE 인가
 거리 지표 · PCA · 변수 선택(제거 / 추가)
 앙상블 - Random Forest 와 Gradient Boosting
 딥러닝의 Linear 층
 모든 숫자는 battery_process_data.csv 로 직접 계산



평가 지표 ① - R² 와 MSE · RMSE 의 공식

R² 는 MSE 를 '데이터의 분산' 으로 나눈 상대값 - 분모가 바뀌면 R² 도 바뀐다

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{\text{MSE}}{\text{Var}(y)}$$

$$\text{MSE} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\text{MSE}} \quad \text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|$$

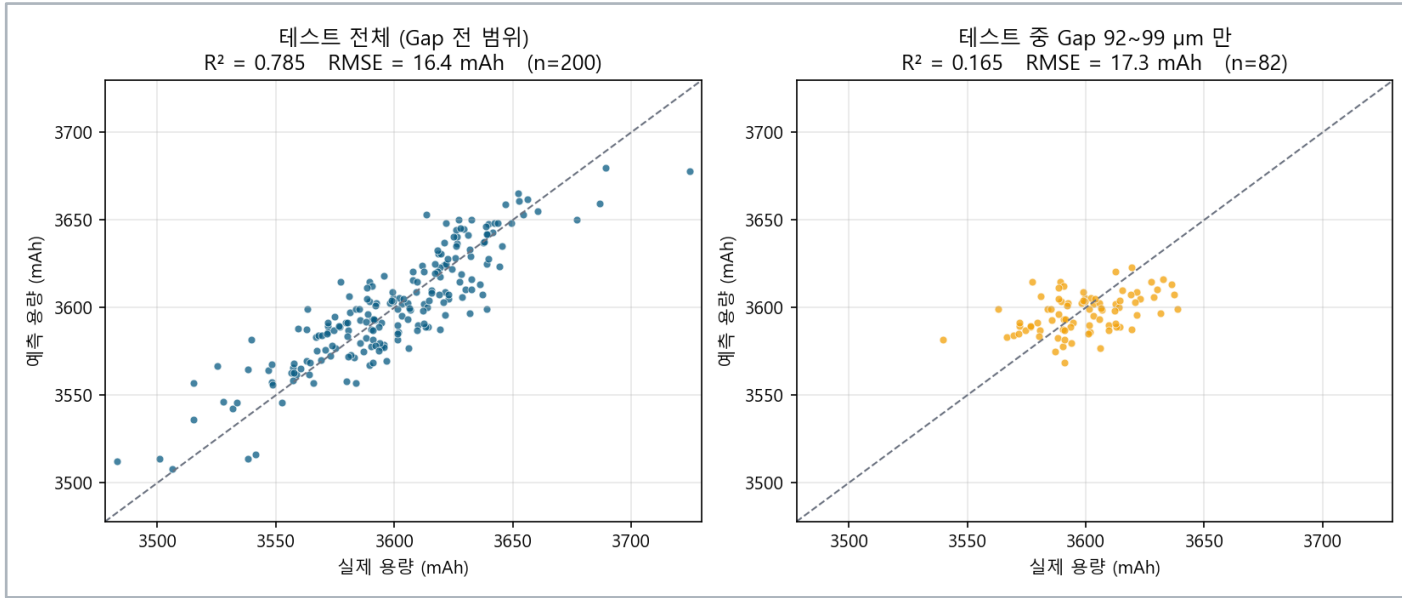
우리 모델 (테스트)	전체 200건	Gap 92~99 μm 82건
MSE (mAh ²)	267.4	300.3
Var(y) (mAh ²)	1243.6	359.8
R ² = 1 - MSE / Var	1 - 267.4 / 1243.6 = 0.785	1 - 300.3 / 359.8 = 0.165
RMSE (mAh)	16.4	17.3
MAE (mAh)	13.2	14.4

[표 T-1] 같은 모델, 평가 범위만 다르게

- 분자(MSE)는 거의 그대로(267 → 300)인데 분모(분산)가 1,244 → 360 으로 줄자 R² 가 0.785 → 0.165 로 떨어졌다
- R² 는 '모델이 얼마나 틀리나' 가 아니라 '데이터가 원래 얼마나 퍼져 있었나' 에 크게 좌우된다
- RMSE = √MSE 는 타겟과 같은 단위(mAh) - "평균 약 16 mAh 틀린다" 로 규격 · 공차와 바로 비교된다

평가 지표 ② - 그래서 요즘은 RMSE(MSE)를 먼저 본다

관리가 잘 된 공정일수록 분산이 작아 R^2 는 낮게 나온다 - 모델 탕이 아닐 수 있다



[그림 T-1] 같은 모델 - 점이 퍼진 폭(오차)은 비슷, R^2 만 크게 다름

이유	설명
단위가 있다	mAh 로 말한다 - 규격 · 공차(예: ± 20 mAh)와 바로 비교
데이터 폭에 덜 흔들림	좁은 범위 · 안정 공정에서도 오차 크기 그대로
학습 목표 그 자체	회귀 · 신경망 손실함수가 MSE - 최적화하는 값을 그대로 보고
비교가 공정함	같은 타깃이면 데이터셋 · 모델이 달라도 RMSE 끼리 비교
R^2 는 보조로	'분산의 몇 % 를 설명' - 음수도 가능, 학습 0.97 / 테스트 0.785

MAE 는 큰 오차에 덜 민감, RMSE 는 큰 오차를 제곱으로 더 별한다 - 큰 불량 한 번이 비싼 현장이면 RMSE

거리 지표 ① - '두 로트가 얼마나 다른가' 를 재는 4가지 자

KNN · K-means · 이상탐지 · 유사 로트 검색이 모두 거리에서 출발한다

$$d_E(a, b) = \sqrt{\sum_j (a_j - b_j)^2}$$

유클리드

직선 거리. 가장 기본 - 큰 차이 하나에 민감

$$d_M(a, b) = \sum_j |a_j - b_j|$$

맨해튼

축을 따라 걷는 거리. 이상치에 덜 민감

$$d_{cos}(a, b) = 1 - \frac{a \cdot b}{\|a\| \|b\|}$$

코사인

방향(패턴)만 본다. 크기는 무시 - 스펙트럼 · 텍스트

$$d_{Mah}(a, b) = \sqrt{(a - b)^T S^{-1} (a - b)}$$

마할라노비스

공분산 S 로 단위 · 상관을 보정 - 공정 이상탐지의 기본

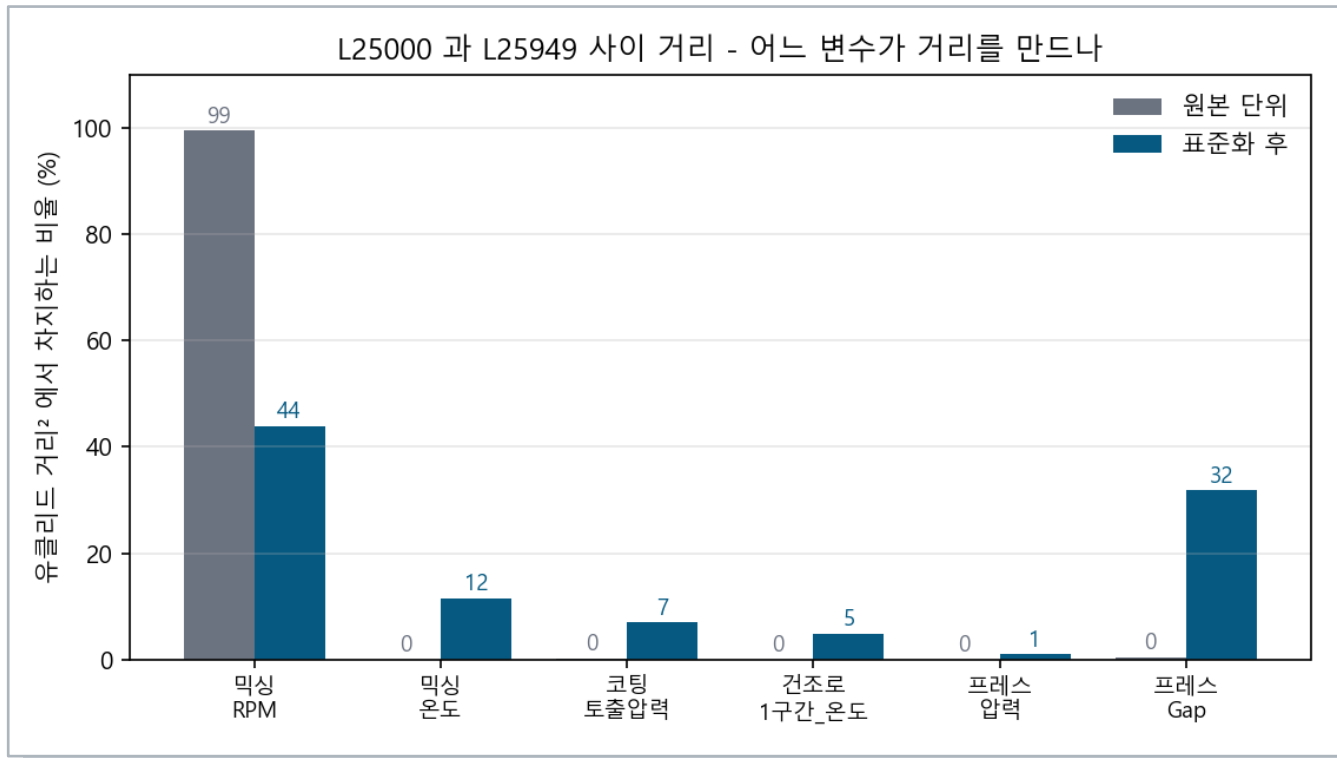
L25000 vs L25949	값
유클리드 (원본 단위)	317.0
유클리드 (표준화 후)	6.03
맨해튼 (표준화 후)	12.97
코사인 거리 (원본)	0.0007
코사인 거리 (표준화 후)	1.31
마할라노비스	5.99

[표 T-2] 정상 로트 vs 불량 로트

L25949: Gap 114.2 · RPM 1458 - 앞서 이상탐지와 불량 예측이 모두 가리킨 로트
원본 코사인 0.0007 = '거의 같은 방향' - RPM 이 벡터를 지배해 차이가 안 보인다

거리 지표 ② - 거리는 단위가 큰 변수가 독차지한다

스케일링(3.5)을 해야 거리가 '공정 차이' 를 낸다



[그림 T-2] 두 로트 사이 유클리드 거리² 의 변수별 몫

$$z_j = \frac{x_j - \mu_j}{\sigma_j}$$

- 원본: 거리 317 의 99.4% 가 믹싱_RPM - Gap 차이 20.7 μm 는 0.4%
- 표준화 후: RPM 44% · Gap 32% - 비로소 Gap 이 보인다
- 마할라노비스 5.99 ~ 표준화 유클리드 6.03 - 이 데이터는 공정 변수끼리 상관이 거의 없어서
- 센서끼리 상관이 크면(온도 구간 1~5 등) 마할라노비스가 훨씬 정직하다
- Isolation Forest 는 거리 대신 '몇 번 잘라야 고립되나' 를 쓴다 - 그래서 스케일에 둔감

PCA ① - 여러 변수를 '분산이 큰 방향' 몇 개로 요약

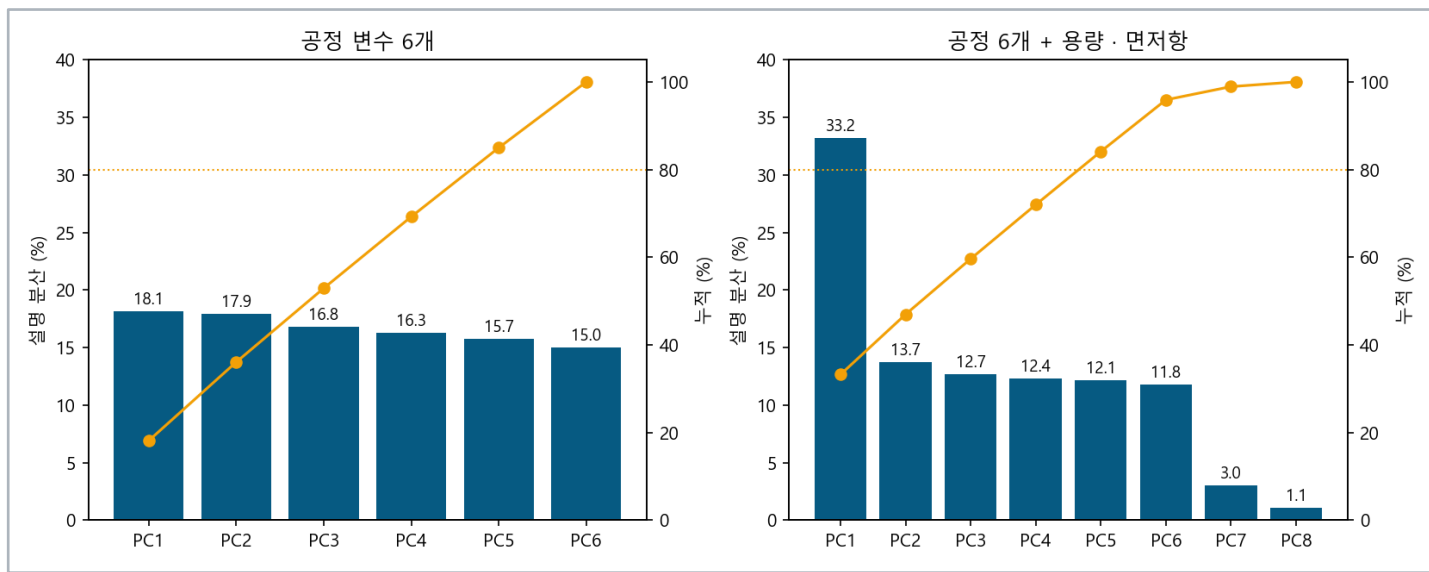
표준화 → 공분산 행렬 → 고유벡터(방향) · 고유값(그 방향의 분산)

$$C = \frac{1}{n-1} Z^T Z,$$

$$C v_k = \lambda_k v_k,$$

$$PC_k = Z v_k$$

$$\text{explained}_k = \frac{\lambda_k}{\sum_j \lambda_j}$$



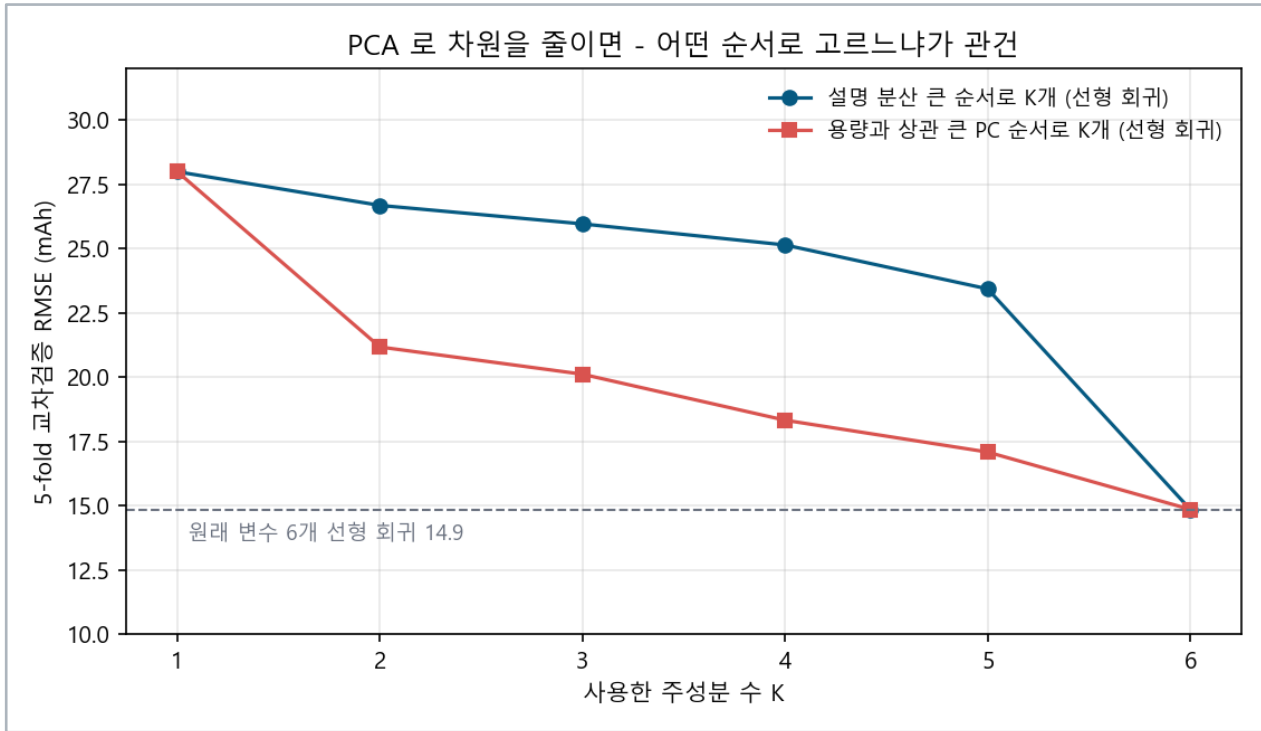
[그림 T-3] 설명 분산 비율 (막대) · 누적 (선)

- 공정 6개: PC1~6 이 15~18% 로 고르다 - 변수끼리 독립이라 요약할 게 없다
- 용량 · 면저항까지 8개: PC1 이 33.2%
- PC1 계수: Gap -0.59 · 용량 +0.57 · 면저항 -0.55 = 'Gap 이 크면 면저항 ↑ 용량 ↓' 한 축
- PCA 가 빛나는 곳: 서로 얽힌 센서 수십~수백 개 (온도 프로파일, 스펙트럼, 설비 태그)

PCA ② - 설명력 높은 주성분만 골라 모델 만들기 (주성분 회귀)

주성분 K 개만 입력으로 쓰고, K 는 교차검증 RMSE 로 정한다

$$\hat{y} = \beta_0 + \sum_{k=1}^K \beta_k PC_k \quad (K < p)$$



[그림 T-4] 주성분 수 K 와 교차검증 RMSE (선형 회귀)

주성분	설명 분산 %	용량과 상관 r
PC1	18.5	+0.65
PC2	17.9	+0.06
PC3	16.7	+0.04
PC4	16.5	-0.02
PC5	15.3	+0.08
PC6	15.0	-0.61

[표 T-3] 학습 데이터 주성분

- 분산 순서로 5개(누적 85%)를 써도 RMSE 23.4 - 6개 14.9 보다 훨씬 나쁘다
- 분산이 가장 작은 PC6(15%)이 용량과 상관 -0.61 - 버리면 안 되는 성분
- PCA 는 y 를 모른다(비지도) - '설명력' 은 x 의 분산이지 예측력이 아니다
- 그래서 K 는 누적 80% 규칙이 아니라 CV RMSE 로 · y 를 같이 보는 PLS 도 대안

변수 선택 ① - 하나씩 빼 가며 최적화 (후진 제거, Backward Elimination)

전부 넣고 시작 → 빼도 RMSE 가 가장 덜 나빠지는 변수를 하나씩 제거

$$S^* = \arg \min_S \text{RMSE}_{CV}(S)$$

- ① 6개 전부로 CV RMSE
- ② 각 변수를 하나씩 빼 본 모델 6개를 비교
- ③ 가장 덜 아픈(RMSE 최소) 변수를 제거
- ④ 더 빼면 확 나빠질 때 멈춘다
- 평가: Random Forest, 학습 데이터 5-fold CV

단계	남은 변수	뺀 변수	CV RMSE	이 단계 후보별 RMSE (뺐을 때)
0	6	-	16.09	-
1	5	건조로_1구간_온도	15.99	믹싱RPM 16.2 믹싱온도 16.2 토출압 18.2 건조 16.0 압력 16.1 Gap 34.3
2	4	믹싱_온도	16.04	믹싱RPM 16.1 믹싱온도 16.0 토출압 18.3 압력 16.1 Gap 34.2
3	3	믹싱_RPM	16.17	믹싱RPM 16.2 토출압 18.3 압력 16.3 Gap 34.7
4	2	프레스_압력	16.47	토출압 19.0 압력 16.5 Gap 35.5
5	1	코팅_토출압력	19.59	토출압 19.6 Gap 37.4

[표 T-4] 후진 제거 과정

프레스_Gap 을 빼면 RMSE 가 34 로 두 배 - 끝까지 남는다 / 건조 · 믹싱 온도는 빼도 오히려 조금 좋아진다(잡음)

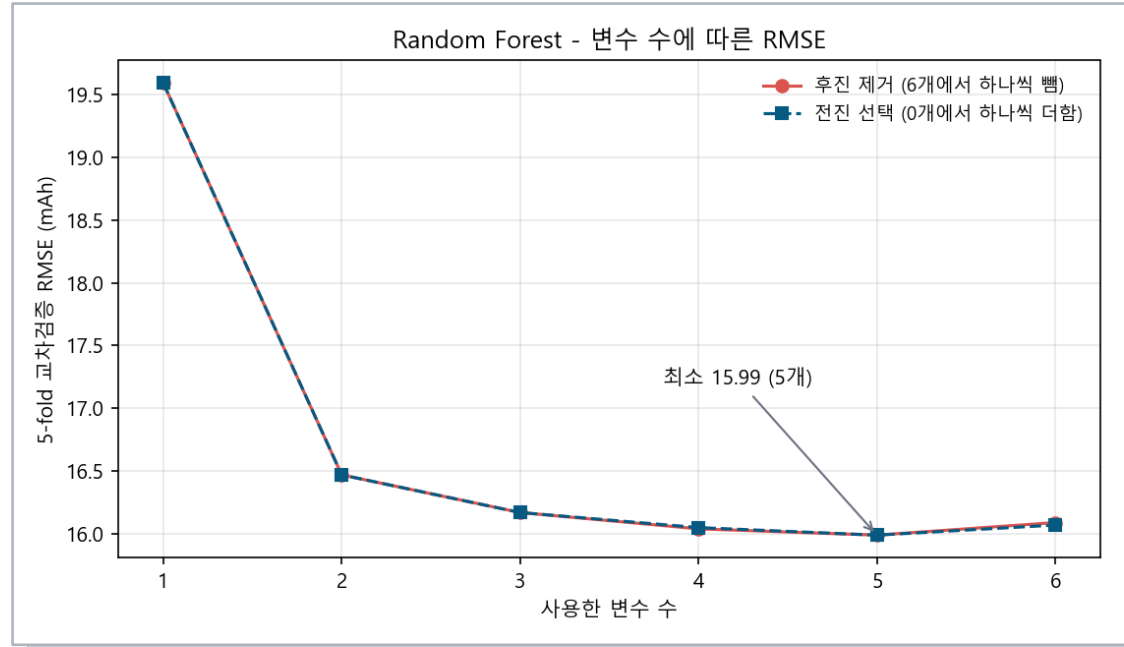
변수 선택 ② - 하나씩 더해 가며 최적화 (전진 선택, Forward Selection)

빈 모델에서 시작 → 넣었을 때 RMSE 가 가장 많이 줄어드는 변수를 하나씩 추가

단계	추가한 변수	CV RMSE	개선
1	프레스_Gap	19.59	-
2	코팅_토출압력	16.47	-3.12
3	프레스_압력	16.17	-0.30
4	믹싱_RPM	16.05	-0.12
5	믹싱_온도	15.99	-0.06
6	건조로_1구간_온도	16.07	+0.08

[표 T-5] 전진 선택 과정

- Gap 하나로 19.6, 토출압력을 더하면 16.5 - 두 개로 거의 다 왔다
- 3개부터는 개선이 0.1~0.3 mAh - '개선 < 기준이면 멈춤' 으로 간단한 모델을 고른다
- 두 방법이 같은 순서에 도착 - 변수끼리 상관성이 크면 결과가 달라질 수 있다



[그림 T-5] 변수 수에 따른 CV RMSE

	전진 선택	후진 제거
시작	0개	전부
계산량	변수 많을 때 유리	변수 적을 때 무난
sklearn	SequentialFeatureSelector(direction='forward')	direction='backward' · RFE

앙상블 ① - 왜 Random Forest 와 Gradient Boosting 을 주로 쓰나

나무 한 그루는 불안정 - 여러 그루를 '평균(배깅)' 하거나 '차례로 보완(부스팅)' 한다

Random Forest = 배깅 + 변수 무작위

$$\hat{f}_{RF}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$$
$$\text{Var}(\hat{f}_{RF}) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2$$

- 나무를 병렬로 깊게, 부트스트랩 표본 + 분기마다 변수 일부만
- 나무끼리 상관 ρ 를 낮춰 분산(흔들림)을 줄인다
- 튜닝 거의 없이 기본값으로 좋다 - 첫 모델로

Gradient Boosting = 잔차를 차례로 학습

$$F_m(x) = F_{m-1}(x) + \eta h_m(x), \quad h_m \approx -\frac{\partial L}{\partial F} = y - F_{m-1}(x)$$

- 얇은 나무를 순서대로, 앞 모델이 틀린 만큼(잔차 = 음의 기울기)을 맞춘다
- 학습률 η 로 조금씩 - 편향(덜 맞춤)을 줄인다
- 튜닝하면 보통 최고 성능 - 대회 · 표 데이터 강자

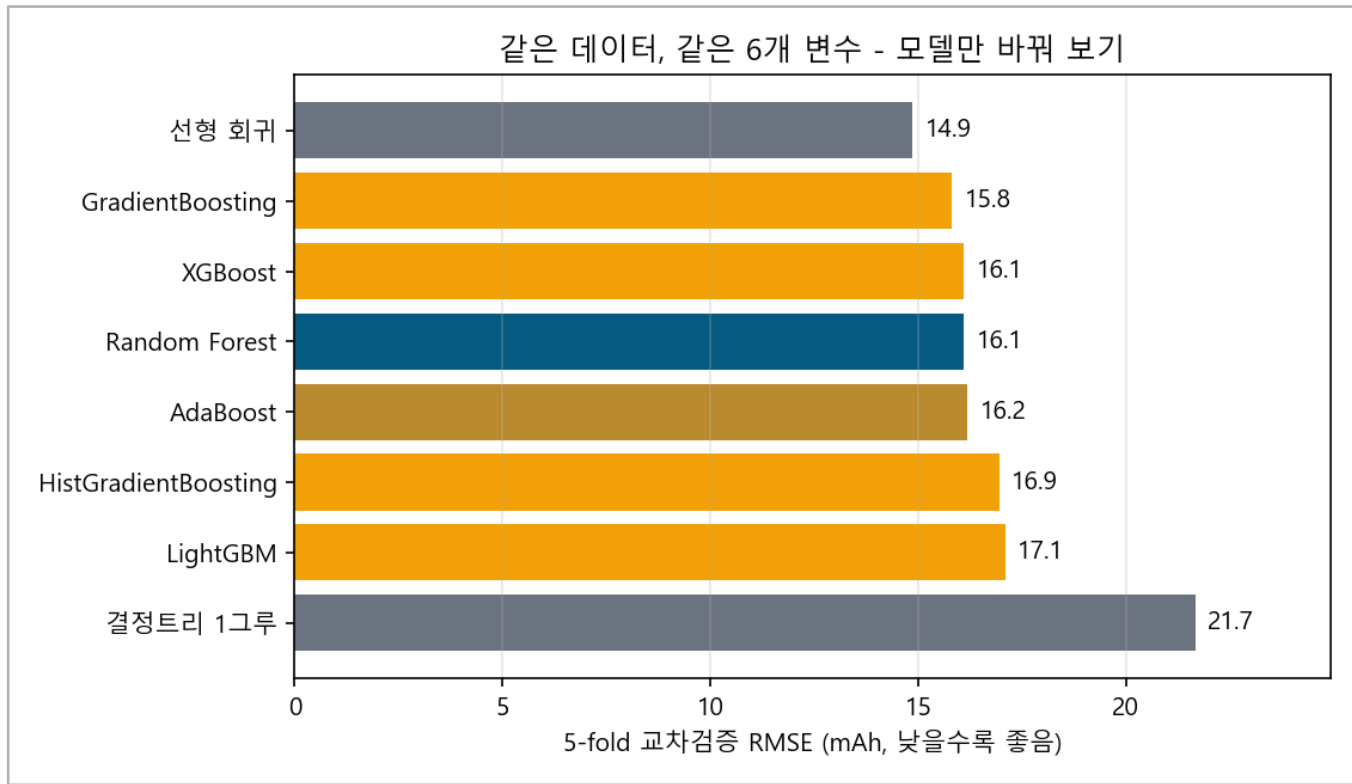
표 데이터(공정 · 품질)에서 둘을 쓰는 이유

비선형 · 변수 상호작용을 자동으로 / 스케일링 불필요 / 이상치 · 치우친 분포에 강함 / 결측 처리 쉬움(HistGB · XGBoost) / 변수 중요도 제공 / 수백~수만 행에서도 잘 됨

참고: Grinsztajn et al., "Why do tree-based models still outperform deep learning on tabular data?", NeurIPS 2022 Datasets and Benchmarks

앙상블 ② - 우리 데이터로 비교하면

같은 6개 변수, 학습 데이터 5-fold 교차검증 RMSE



[그림 T-6] 모델별 CV RMSE (mAh)

- 나무 1그루 21.7 → Random Forest 16.1 : 여러 그루 평균으로 오차 26% 감소
- 부스팅 계열(GB · XGBoost · LightGBM · HistGB)은 15.8~17.1 - 기본값끼리는 비슷, 튜닝으로 갈린다
- 이 데이터는 선형 회귀가 14.9 로 가장 좋다 - 용량이 Gap 등에 거의 직선으로 반응하게 만든 실습 데이터라서
- 교훈: 앙상블이 늘 이기지 않는다 - 선형 회귀를 기준선 (baseline)으로 먼저 돌려 보고 비교
- 실제 공정은 비선형 · 상호작용 · 구간 효과가 흔해 RF · GB 가 기본 후보가 된다

Gradient Boosting 의 종류 - 빠대는 같고 '빠르게 · 덜 과적합' 경쟁

모두 잔차를 나무로 차례로 맞추는 방식, 차이는 나무 만드는 법과 정규화

$$\text{Obj} = \sum_i L(y_i, \hat{y}_i) + \sum_m (\gamma T_m + \frac{1}{2} \lambda \|w_m\|^2)$$

이름	발표	핵심 아이디어	언제	CV RMSE
AdaBoost	Freund · Schapire (1997)	틀린 샘플 가중치 ↑ - 부스팅의 원조	개념 이해 · 작은 데이터	16.2
GBM (sklearn GradientBoosting)	Friedman (2001)	손실의 음의 기울기(잔차)를 나무로 적합	수천 행, 기본기	15.8
XGBoost	Chen · Guestrin (2016)	정규화 목적식($\gamma \cdot \lambda$) · 2차 미분 · 결측 방향 학습 · 병렬	가장 널리 쓰임 · 대회	16.1
LightGBM	Ke et al. (2017)	히스토그램 분할 · 잎 우선(leaf-wise) 성장 · GOSS · EFB	수십만 행 이상, 빠름	17.1
CatBoost	Prokhorenkova et al. (2018)	Ordered boosting · 범주형 변수 자동 처리 · 대칭 나무	범주형(설비 · 라인 · 자재) 많을 때	-
HistGradientBoosting	scikit-learn (0.21~)	LightGBM 식 히스토그램 · 결측값 자체 처리	sklearn 안에서 빠르게	16.9

[표 T-6] Gradient Boosting 계열 (CV RMSE 는 우리 데이터 · 기본에 가까운 설정, CatBoost 는 미설치)

공통 튜닝 3종: 나무 수($n_estimators$) × 학습률($learning_rate$) × 깊이(max_depth / num_leaves) - 학습률을 낮추면 나무를 늘리고 조기 종료(early stopping)로 멈춘다

딥러닝의 Linear 층 ① - $y = Wx + b$, 선형 회귀와 같은 식

PyTorch nn.Linear · Keras Dense = 입력에 가중치를 곱해 더하고 편향을 더한다

$$y = Wx + b \quad (\#params = n_{in} \times n_{out} + n_{out})$$

```
import torch.nn as nn

# 공정 변수 6개 → 용량 1개
net = nn.Linear(6, 1)      # W: 1×6, b: 1 → 7개
loss = nn.MSELoss()        # 손실 = MSE

# 불량 확률이면 끝에 Sigmoid
clf = nn.Sequential(nn.Linear(6, 1),
                    nn.Sigmoid()) # = 로지스틱 회귀
```

- Linear 1층 + MSE 손실 = 선형 회귀 그 자체 - 경사하강법으로 푸느냐, 공식으로 푸느냐만 다르다
- 우리 데이터: nn.Linear(6,1) CV RMSE 14.86 = sklearn 선형 회귀 14.86
- Linear + Sigmoid + 이진 교차엔트로피 = 로지스틱 회귀(불량 확률)
- 파라미터 수 = 입력 × 출력 + 출력 - 층이 넓을수록 빠르게 는다
- 입력은 반드시 스케일링 - 경사하강법은 단위가 다르면 느리고 불안정 (3.5)

딥러닝의 Linear 층 ② - 쌓기만 하면 여전히 직선, 활성화가 곡선을 만든다

Linear 사이에 ReLU 같은 비선형 함수가 있어야 '깊은' 의미가 생긴다

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = W'x + b'$$

$$h = \text{ReLU}(W_1x + b_1), \quad \hat{y} = W_2h + b_2$$

모델 (PyTorch, 표준화 입력)	구조	파라미터	학습 반복	CV RMSE
nn.Linear(6,1)	6 → 1	7	3000	14.86
Linear 2층 (활성화 없음)	6 → 32 → 1	257	3000	14.86
MLP 100 epoch	6 → 32 → 16 → 1, ReLU	769	100	15.84
MLP 3000 epoch	6 → 32 → 16 → 1, ReLU	769	3000	26.20

[표 T-7] 우리 데이터 5-fold CV (Adam, 학습률 0.01)

- 활성화 없는 2층: 파라미터 257개인데 결과가 1층과 똑 같다 - 위 공식처럼 한 층으로 합쳐진다
- ReLU MLP 는 오래 학습할수록 나빠진다(15.8 → 26.2)
- 학습 640행에 파라미터 769개, 과적합
- 그래서 검증 데이터 + 조기 종료 · 드롭아웃 · 가중치 감소가 필수
- 수백~수천 행 표 데이터는 선형 · RF · GB 가 먼저, 딥러닝은 이미지 · 시계열 · 대용량에서

이론 보충 정리 - 질문별로 무엇을 보나

모든 값은 이 실습 데이터로 계산한 결과

질문	쓰는 것	이 데이터에서
모델이 얼마나 틀리나	RMSE · MAE (R^2 는 보조)	RMSE 16.4 mAh, R^2 는 범위 따라 0.785 ↔ 0.165
두 로트가 얼마나 다른가	표준화 유클리드 · 마할라노비스	원본 거리는 RPM 이 99% · 표준화 후 6.03
변수를 요약할 수 있나	PCA 설명 분산	공정 6개는 각 15~18% - 요약 여지 적음
주성분으로 모델을 줄이면	PCR, K 는 CV 로	분산 큰 5개 23.4 vs 6개 14.9 - 작은 PC 가 중요
어떤 변수가 필요한가	후진 제거 · 전진 선택	Gap → 토출압력 → 프레스압력 순, 3개면 충분
어떤 모델을 쓰나	선형 기준선 → RF → GB 계열	선형 14.9 · GB 15.8 · RF 16.1
딥러닝이면	Linear + 활성화 + 조기 종료	Linear = 선형 회귀, MLP 는 과적합 주의

[표 T-8] 이론 보충 요약

다시 계산하기: Simple 에서 python build/make_theory_figs.py (수식 · 그림 · theory.json 생성) → python build/build_extra_gui_deck_qt.py

부록 - 배포 파일과 강사용 정답 앱

수강생: prompts_qt.md / 강사: 정답 앱(파일을 기능별로 나눠 둬)

경로 (SecondaryBattery-AI-Example/번외_Gemini_GUI/)	내용
prompts_qt.md · img/	프롬프트 14개 + 목표 화면(수강생 배포)
main_qt.py · gui_qt.py	정답 앱 - Bootstrap 사이드바 화면, 메뉴 7개(python main_qt.py)
data_io.py · cleaning.py · model.py · charts.py	정답 앱 - 계산(읽기 · 정제 · 모델 · 이상탐지 · 차트 · 리포트)
check_data.py · check_model.py · check_charts.py	강사용 숫자 확인 스크립트
../data/battery_process_data.csv	실습 데이터(교재 부록 A)

[표 X-5] 배포 파일

- 수강생 결과물은 app.py 한 파일이어도 된다 - 정답 앱은 설명하기 쉽게 기능별 파일로 나눠 두었을 뿐
- 목표 화면 다시 만들기: Simple 에서 python build/capture_extra_gui_qt.py → python build/build_extra_gui_deck_qt.py